- Load the Sequel library
- Create a database connection
- Create an expenses table so we have a place to store our records
- Insert records into the expenses table
- Query records from the expenses table

That's quite a task list, but the APIs to perform these steps are simple. In fact, the first example from the Sequel homepage provides a short code snippet that shows how to do all of these![2] If you're following along, we encourage you to launch an IRB session and try all these steps based on the example:

```
>> require 'sequel'
=> true
>> DB = Sequel.sqlite
=> #<Sequel::SQLite::Database: {:adapter=>:sqlite}>
>> DB.create_table(:gems) { String :name }
=> nil
>> DB[:gems].insert(name: 'rspec')
=> 1
>> DB[:gems].insert(name: 'sinatra')
=> 2
>> DB[:gems].all
=> [{:name=>"rspec"}, {:name=>"sinatra"}]
```

Now that you're familiar with the APIs, you're ready to create some databases.

Creating a Database

You'll want to create separate SQLite databases for testing, development, and production (so that you don't clobber your real data during testing).

An environment variable is the easiest way to configure separate databases for this app. In *Getting Started*, on page 47, you used the RACK_ENV variable to indicate which environment the code was running in. Let's piggyback off that work. Create a new file called config/sequel.rb with the following code:

06-integration-specs/03/expense_tracker/config/sequel.rb

```
require 'sequel'
DB = Sequel.sqlite("./db/#{ENV.fetch('RACK_ENV', 'development')}.db")
```

This configuration will create a database file such as db/test.db or db/production.db depending on the RACK_ENV environment variable. Note that you're assigning the database connection to a top-level DB constant; this is the Sequel convention when there's just one global database.[3]

2. http://sequel.jeremyevans.net
3. http://sequel.jeremyevans.net/rdoc/files/doc/opening_databases_rdoc.html